# Klausur Datenbanksysteme

Name: _____

Matrikel-Nr.: _____

Aufgabenblatt **nicht** vor Beginn der Prüfung umdrehen!

| | |
|---|---|
| Prüfer: | Prof. Dr. Martin Hulin |
| Dauer: | 90 Minuten gesamt |
| Datum: | 6. Juli 2010 |
| Hilfsmittel: | A = Alle auf dem Prüfungsplan angegebenen Hilfsmittel B - K |
| Kennzahlen: | 3619 |

**Rückseiten sind bedruckt!**

**Lösen** Sie die Aufgaben auf den **ausgegebenen Aufgabenblättern!**
**Ersatz-Aufgabenblätter** sind auf Anfrage erhältlich.

**Bewertung:**

| **Aufg.** | 1 (30) | 2 (4) | 3 (15) | 4 (8) | 5 (21) | 6 (9) | | Σ (87) |
|---|---|---|---|---|---|---|---|---|
| **Punkte** | | | | | | | | |

**Aufgaben** **(Punkte ohne Gewähr)**

**Aufgabe 1: Erstellen eines Entity-Relationship-Diagramms (oder UML-Klassendiagramms) (30)**

Im Landkreis Ravensburg gibt es ein Netz von ausgeschilderten Radrouten. Diese sollen in einer Datenbank erfasst werden. Erstellen Sie dazu eine Datenbankstruktur als UML-Klassendiagramm (mit Klassen, Attributen und Assoziationen) oder Entity-Relationship-Diagramm (mit Entity-Sets, Attributen und Relationen). Verwenden Sie dazu das ausgeteilte **Extrablatt**. Achten sie darauf, ...

- dass Sie ein ER-Diagramm (UML-Diagramm) erstellen sollen und nicht ein Diagramm einer relationalen Tabellenstruktur, d.h. M:N-Relationen sind noch nicht in Verbindungstabellen umgewandelt, es gibt mehrwertige Attribute, und eine Strukturierung in Unter-Entities (Unterklassen) ist möglich.
- dass Sie Anomalien und Redundanzen vermeiden.
- dass Sie die Art der Relationen in der ausführlichen **(min, max)-Schreibweise** kennzeichnen, falls Sie ein ER-Diagramm zeichnen.
- Achten Sie auch auf die SQL-Anweisungen in den Aufgaben 2, 3 und 5a. Diese Abfragen sollen mit der Datenbank-Struktur möglich sein.

**Beschreibung der Datenbank für Radrouten und Wegweiser**

Unten stehende Informationen sollen in der Datenbank enthalten sein und effizient abgefragt werden können. Informationen, die hier nicht genannt sind, brauchen auch nicht ins ER-Diagramm, selbst wenn Sie sie für eine Datenbank für Rad-Wegweiser für sinnvoll erachten.

- Im Landkreis Ravensburg gibt es ein Netz von **Radrouten**. Dabei können sich Routen auch kreuzen oder ein kurzes Stück gemeinsam laufen. Unten sehen Sie einen Kartenausschnitt mit einigen Routen, z. B. *Route G* von *Wangen* nach *Weingarten*.

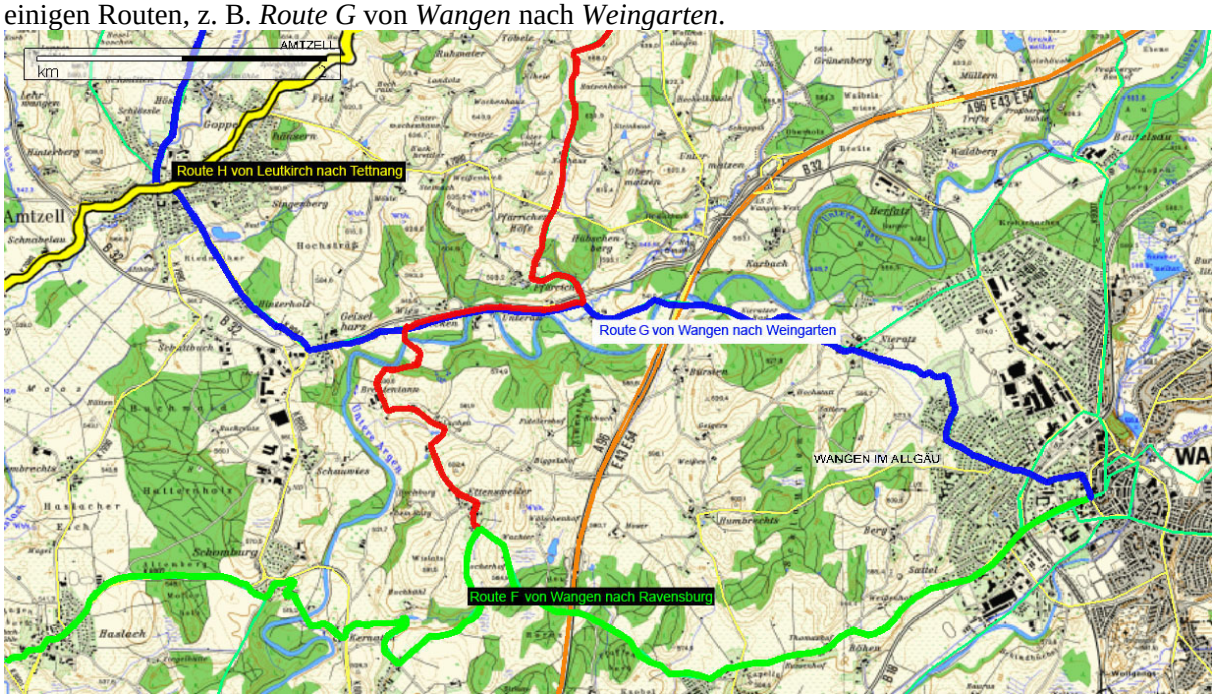


**Abbildung 1: Beispiel-Routen**

- Die Radrouten sind mit besonderen **Wegweisern** ausgeschildert. Die Wegweiser sind an bestimmten **Standorten** befestigt, vorzugsweise Kreuzungen oder Abzweigungen. Damit die Bauhöfe der Gemeinden wissen, wo sie die Wegweiser aufstellen sollen, wird die Lage der Standorte und die Befestigungsmöglichkeit der Wegweiser daran in der Datenbank beschrieben. Außerdem werden die GPS-Koordinaten des Standorts angegeben.

  Bei jedem Standort will man auch wissen, wie viele **km** er jeweils vom Startpunkt der Routen entfernt ist.

  An den Standorten sind ein oder zwei oder mehrere (wenn sich an dem Standort mehrere Routen kreuzen) Wegweiser angebracht. Zwei Beispiele sehen Sie auf den Fotos unten. Die Radwegweiser sind weiß mit grünem Aufdruck.

  Es gibt zwei Sorten von Wegweisern: Auf der Abbildung 2 sind **Hauptwegweiser** zu sehen. Sie haben zwei Zeilen mit einem Fernziel in der ersten Zeile und einem Nahziel in der zweiten Zeile, dahinter die zugehörigen km-Angaben. Auf der Abbildung 3 sind **Zwischenwegweiser** zu sehen. Sie haben nur einen Pfeil aufgedruckt. Dieser Pfeil kann geradeaus ↑, links ←, rechts →, schräg links ↖, schräg rechts ↗, vorne links ↰ oder vorne rechts ↱ zeigen.

- Bei regelmäßigen Kontrollfahrten werden die Standorte kontrolliert. Dabei werden von einem Standort ein oder mehrere **Fotos** aufgenommen und in der Datenbank gespeichert. Dadurch können die Bauhöfe der zuständigen Gemeinden einfach erkennen, welche Fehler bei den Standorten vorliegen. Manchmal sind auf einem Foto auch zwei nahe beieinander liegende Standorte zu sehen.

- Bei den Kontrollfahrten werden die Fehler oder Bemerkungen zu den Standorten von den Kontrolleuren auch als **Kommentar** in die Datenbank eingetragen.

**Abbildung 2: Standort mit Hauptwegweisern** **Abbildung 3: Standort mit Zwischenwegweisern**

- Die Gemeinden wollen Fehlerlisten aus der Datenbank generieren können, in der alle Standorte mit fehlenden oder fehlerhaften Wegweisern aufgeführt sind, für die sie zuständig sind.
- Die Information zu einem Standort (hier Standort *F04* an der *Route F Wangen – Ravensburg*) könnte so aussehen (siehe Abbildung 4):

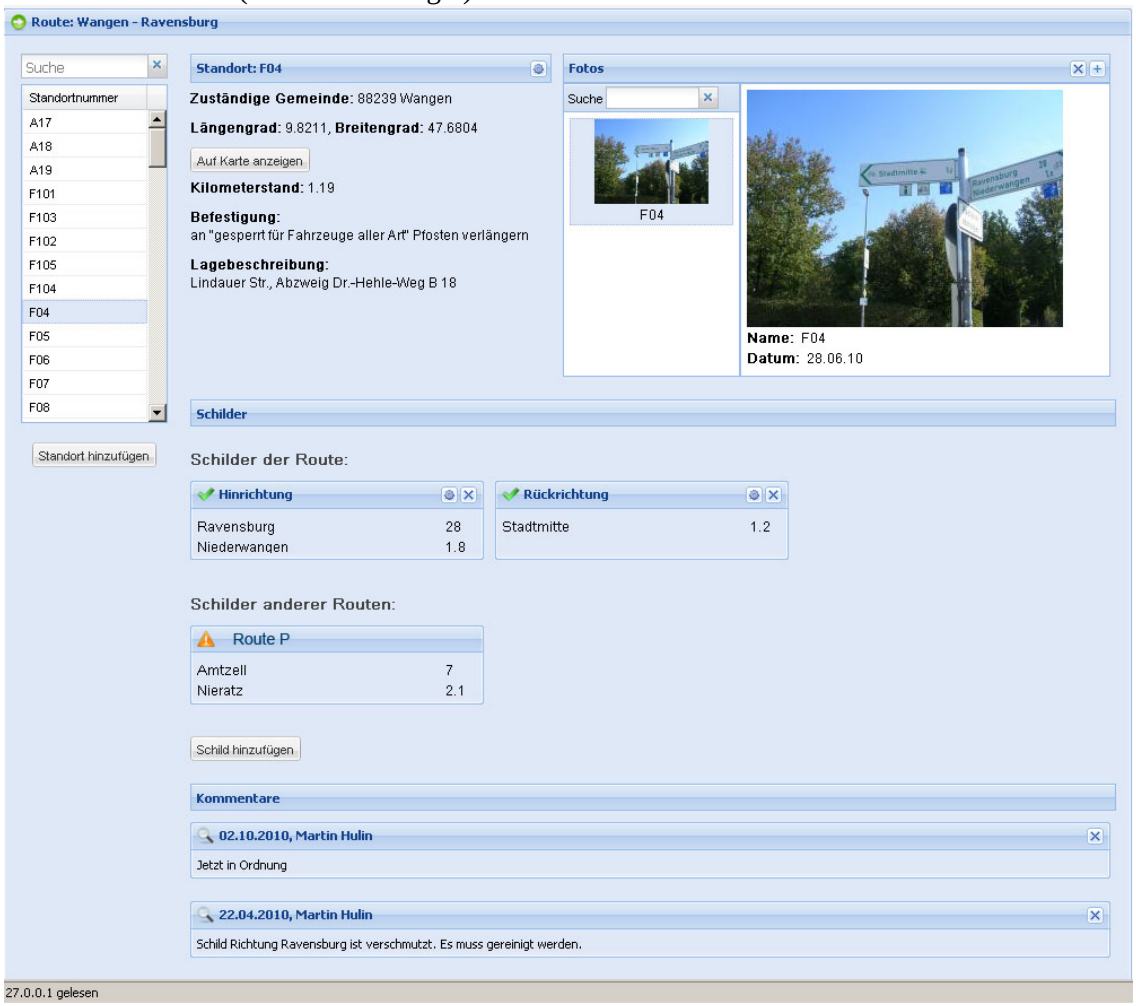


**Abbildung 4: Screen-Shot der Web-Anwendung zur Verwaltung von Rad-Wegweisern**

**Wenn Sie die Aufgabe 1 gelöst haben, geben Sie bitte das Extrablatt mit Ihrer Lösung ab. Sie erhalten dafür die Musterlösung, um die folgenden Aufgaben bearbeiten zu können.**

**Aufgabe 2: SQL** **(4)**

Geben Sie eine SQL-Abfrage an, die die *WId* aller Wegweiser ermittelt, bei denen der Zustand '*fehlt*' oder *'mangelhaft'* ist.

**Aufgabe 3: SQL** **(15)**

Radfahrer beschweren sich, dass an der *Kreuzung der L304 mit der K3740* kein Wegweiser vorhanden ist. Daher wird für die Route mit der RId *15* (schon in der Datenbank vorhanden) von der zuständigen Gemeinde mit der GId *30* (schon in der Datenbank vorhanden) ein neuer Standort mit der SId *'F17'* an dieser Kreuzung mit den GPS-Koordinaten (*9,79°; 47,773°*) beim km-Stand *25,2* eingefügt. Am *Vorfahrtsschild rechts* soll dort ein Zwischenwegweiser geradeaus ↑ in Richtung Ravensburg (Hin-Richtung) mit der WId *333* angebracht werden.

Geben Sie alle SQL-Anweisungen an, um diese Informationen in der Datenbank zu speichern. Schließen Sie auch die Transaktion ab.

**Aufgabe 4: Normalformen, funktionelle Abhängigkeiten** **(8)**

Auf den Wegweisern sollen Piktogramme aufgedruckt werden können, die auf besondere Einrichtungen wie Bahnhof, Werkstatt oder Tourist-Information hinweisen. Pro Wegweiser kann nur ein Piktogramm gedruckt werden. Der Student Otto Znadnuder legt für die erweiterte Wegweiser-Datenbank folgende Tabelle **WegweiserZnadnuder** an (hier mit vier Beispiel-Datensätzen):

| WId | Typ | Zustand | Richtung | Fernziel | km-fern | Nahziel | km-nah | Pfeil-richt. | RId | SId | Pikto | Bedeutung | Bild |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 10 | Haupt | fehlt | hin | Wangen | 15 | Amtzell | 5,1 | | 15 | F05 | I | Information | |
| 11 | Haupt | o.k. | rück | Wein-garten | 16 | Wolfegg | 4,2 | | 15 | F05 | B | Bahnhof | |
| 12 | Haupt | o.k. | rück | Wein-garten | 15 | Wolfegg | 3,3 | | 15 | F06 | B | Bahnhof | |
| 13 | ZW | o.k. | hin | | | | | links | 15 | F124 | | | |

Ist die Spalte **WId** Superschlüssel der Tabelle?
☒ Ja ☐ Nein✓

Ist die Spalte **Pikto** Superschlüssel der Tabelle?
☐ Ja ☒ Nein✓

Geben Sie eine funktionelle Abhängigkeit an, die der Boyce-Codd Normalform widerspricht.

Teilen Sie die Tabelle so auf, dass die entstehenden Tabellen in Boyce-Codd Normalform sind. Sie brauchen nur die Struktur der entstehenden Tabellen hinschreiben, nicht die Datensätze

**Aufgabe 5: SQL, Programmieraufgabe**

a) Gegeben ist ein Parameter für die Route in der Variablen `:rid`. Geben Sie eine SQL-Abfrage an, die für diese Route für alle Standorte, die auf dieser Route liegen, die Daten: *Länge*, *Breite* und *km* (Entfernung des zugehörigen Standorts vom Start der Route) sortiert nach *km* ermittelt. **(6)**

b) Schreiben Sie eine C-Funktion mit Embedded SQL oder eine PHP-Funktion `StandortDistanz,` die auf die Wegweiser-Datenbank zugreift. Als Parameter hat die Funktion die Zahl `RouteId` (bzw. bei PHP `$RouteId`). Die Funktion soll eine Übersicht aller Standorte auf dieser Route zusammen mit dem Abstand zwischen zwei aufeinanderfolgenden Standorten ausdrucken. Das Ergebnis soll so aussehen: **(15)**

Standorte Route 15

| km-Differenz | Länge | Breite |
|---|---|---|
| | 9.8211 | 47.6804 |
| 0,25 | | |
| | 9.829 | 47.6753 |
| 0,35 | | |
| | 9.8313 | 47.6701 |
| 0,6 | | |
| | unbekannt | |
| 1,8 | | |
| | 9.845 | 47.6503 |
| … | | |

**Hinweise:**

- Die Datenbankverbindung ist bereits eingerichtet. Bei PHP kann die Datenbankverbindung über die globale Variable `$dblink` referenziert werden. Bei embedded SQL ist die SQLCA (SQL Communication Area) bereits eingebunden.
- Das Feld *km* in der Tabelle *liegt_auf* hat ein NOT NULL Constraint, ist also immer belegt. Dagegen müssen *Länge* und *Breite* nicht unbedingt belegt sein.
- Eine Fehlerbehandlung braucht nicht programmiert werden.
- Die Abfrage von a) brauchen Sie nicht nochmal hinzuschreiben.

```
void StandortDistanz (int RouteId) {
```

bzw.

```
function StandortDistanz ($RouteId) {
```

**Aufgabe 6: Sekundärindex** **(9)**

Für die Tabelle *Wegweiser* wird für die Spalte *Nahziel* ein **Sekundärindex** (kein B-Baum) eingerichtet. Folgende Fakten sind gegeben:

- Es gibt 20400 Wegweiser in der Datenbank.
- Jeder Wegweiser-Datensatz benötigt 80 Byte.
- Ein Block kann 8192 Byte speichern
- Die Spalte Nahziel ist 20 Byte lang.
- Ein Blockzeiger ist 6 Byte groß.
- Es gibt 5 Wegweiser mit dem Nahziel 'Nieratz', die in verschiedenen Datenbank-Blöcken gespeichert sein können.
- Sowohl die Datenblöcke als auch die Indexblöcke sind maximal gefüllt, außer evtl. dem letzten Block; es gibt keine halb leeren Blöcke.

a) Wie viele Blockzugriffe benötigt man maximal, wenn man **alle** Wegweiser mit dem Nahziel 'Nieratz' finden will und **keinen** Index verwendet?

b) Wie viele Blockzugriffe benötigt man maximal, wenn man **alle** Wegweiser mit dem Nahziel 'Nieratz' finden will **und dabei den Sekundärindex für die Spalte Nahziel nutzt**.

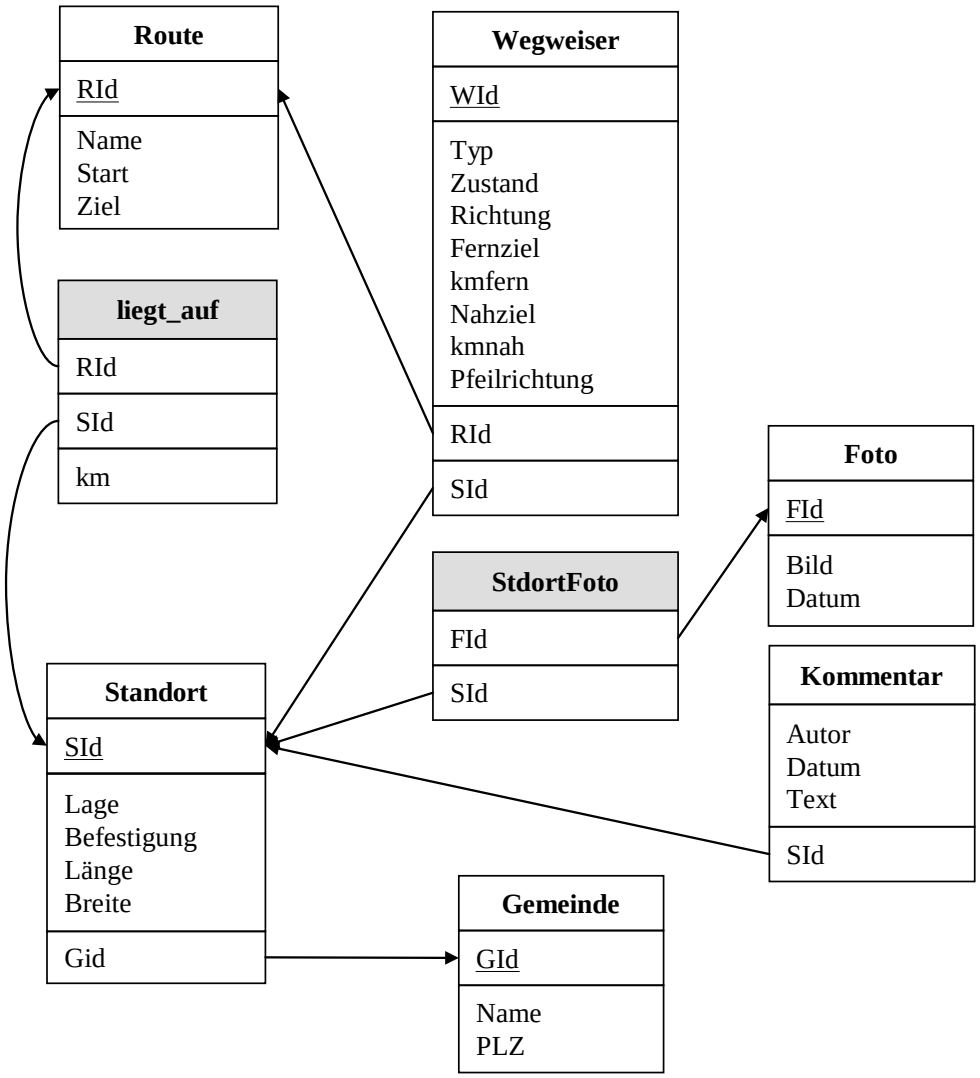


**Abbildung 5: Tabellenstruktur der Datenbank für Rad-Wegweiser**

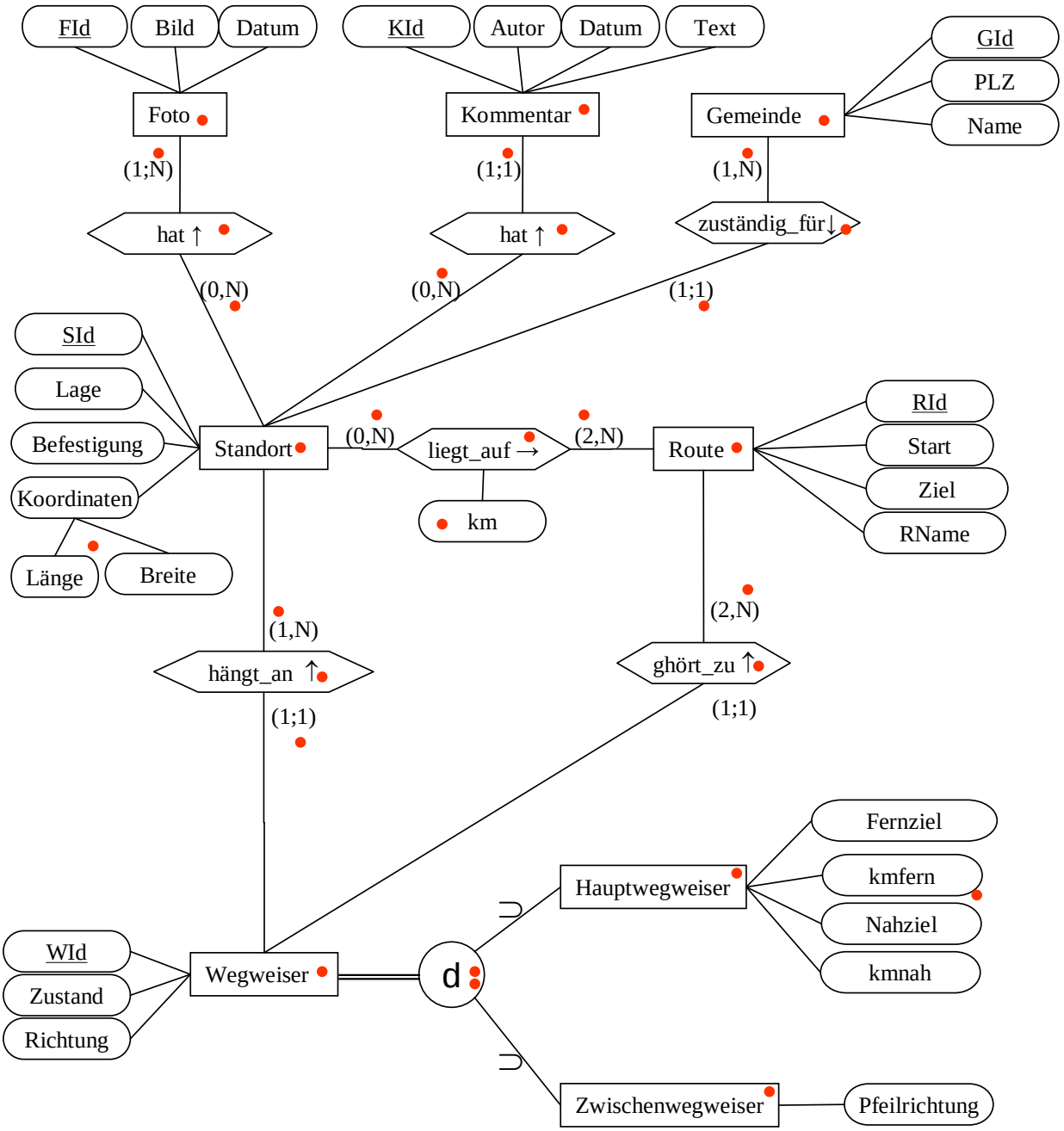


**Abbildung 6: Entity-Relationship-Diagramm der Datenbank für Rad-Wegweiser**